



MULTI-AGENT COORDINATOR

A Multi-Agent System for Automated Full-Stack Software Generation

PROJECT DOCUMENTATION REPORT

Repository: github.com/abdallared/multi-agent-swe · v1.0

Submitted to: Course Instructor

Date: July 2026

Table of Contents

- 1. Executive Summary 3
- 2. Introduction 3
- 3. System Architecture 4
- 4. The Agents 5
- 5. Supporting Modules 6
- 6. Technology Stack..... 6
- 7. Project Structure 7
- 8. Key Features 8
- 9. Installation & Usage Workflow 8
- 10. Testing.....10
- 11. Results & Performance.....11
- 12. Documentation Set.....11
- 13. Limitations & Future Work12
- 14. Conclusion13
- 15. Reference.....13

1. Executive Summary

Multi-Agent Coordinator started from a fairly simple question: could a team of small, specialized AI agents replace the repetitive first hour of building a full-stack web app? Rather than handing one large prompt to a single model and hoping for a working project, the system splits the work into four roles — planning, architecture, backend development, and frontend development — and gives each one to its own dedicated agent. The agents run one after another in a fixed pipeline, each picking up exactly where the previous one left off, and by the end of the run there is a working codebase on disk: authentication, a database, an API, and a working web interface.

Everything runs locally through Ollama with the llama3.2 model, so there is no API bill involved and no dependency on an internet connection during generation. The project can be used two ways — a command-line tool for quick, no-frills generation, and a full web application (FastAPI backend, React/TypeScript frontend) for anyone who wants to watch the pipeline work in real time.

Based on the testing logged while building this system, the pipeline produces a working project roughly 93% of the time, takes about five to ten minutes per run, and — after a round of reliability fixes described later in this report — no longer requires any manual patching before the generated application actually runs.

Attribute	Value
Project name	Multi-Agent Coordinator
Repository	github.com/abdallared/multi-agent-swe
Version	1.0
License	MIT License
Primary language	Python (77.4%), TypeScript (15.9%)
Success rate	≈ 93%
Generation time	5–10 minutes per project
Inference cost	Zero (local Ollama model)

2. Introduction

2.1 Motivation

Anyone who has built more than a couple of full-stack applications knows the first few hours look almost the same every time: define the data model, wire up a REST API, bolt on authentication, then build a frontend that talks to it. It is necessary work, but it is also the part of the job that benefits the least from human judgment — it is mostly pattern-following. That repetitiveness is what made me want to test whether a coordinated group of agents, each responsible for one stage of the process, could take over the scaffolding reliably. The idea mirrors

how a small human dev team already splits the work between a PM, an architect, a backend engineer, and a frontend engineer — the pipeline here just assigns each of those roles to a dedicated agent instead of a person.

2.2 Objectives

- Automate the generation of complete full-stack projects — backend, frontend, database, and authentication — starting from a plain-language project description.
- Demonstrate a multi-agent architecture where every agent has one clear, well-defined responsibility rather than one model trying to do everything at once.
- Keep the whole system free to run by using a locally hosted LLM (Ollama) instead of a paid cloud API.
- Offer two usable interfaces: a CLI aimed at developers, and a web UI for a broader, less technical audience.
- Produce output that is immediately runnable — a generated project should install and start with standard tooling (pip, npm, uvicorn, vite) without any manual fixes.

2.3 Scope

Right now the system focuses on one common, well-understood pattern: a FastAPI + SQLAlchemy backend paired with a React + TypeScript + Tailwind CSS frontend, using SQLite for storage and JWT-based authentication. I picked this combination deliberately — it is broad enough to cover a lot of real-world CRUD-style applications (task managers, booking systems, inventory tools, and similar) while staying narrow enough that the agents can reliably generate correct, consistent code instead of guessing at an unfamiliar stack each time.

3. System Architecture

The system is organized as a pipeline of specialized agents coordinated by a central orchestrator — `main.py` handles this for the CLI, and the FastAPI app plays the same role for the web UI. Every agent inherits from a shared `BaseAgent` class and talks to the local LLM through a common configuration layer, so adding or adjusting an agent does not mean re-plumbing the whole system.

3.1 High-Level Pipeline

User Input — *project name, description, requested features*



Planner Agent — *analyzes requirements, defines features*



Architect Agent — *designs system architecture & API contracts*



Backend Agent — *generates FastAPI + SQLAlchemy backend code*



File Builder — writes the generated code to disk as a real project



Frontend Agent — generates React + TypeScript + Tailwind frontend



Output — a complete, runnable full-stack project

3.2 Two Operating Modes

CLI Mode

```
main.py → Planner → Architect → Backend → Builder → Frontend → output/
```

Run directly from the terminal with `python main.py`. Generated projects land in the `output/` directory.

Web UI Mode

```
ui/frontend (React) → ui/backend/app.py (FastAPI + WebSocket) → Agents → ui/backend/output/
```

The web interface lets someone type a project name, description, and desired features into a form, then watch generation progress update live over a WebSocket connection. Generated projects are written to `ui/backend/output/`, and logs go to `ui/backend/logs/`.

3.3 Architectural Style

The design follows a pipeline / sequential multi-agent pattern: each agent consumes the structured output of the previous agent and produces structured output for the next one. This is simpler and more predictable than a fully autonomous multi-agent negotiation setup, which fits a system whose real goal is reliable, repeatable code generation rather than open-ended reasoning.

4. The Agents

All agents live in the `agents/` directory (7 files total) and share a common base class, `base_agent.py`, which defines the interface every agent has to implement: an `execute()` method that carries out the agent's task, and a `get_system_prompt()` method that defines the agent's role and instructions to the underlying LLM.

Agent	File	Responsibility
Base Agent	<code>base_agent.py</code>	Shared base class defining the <code>execute()</code> / <code>get_system_prompt()</code> interface used by every agent.
Planner	<code>planner.py</code>	Analyzes the user's project description and determines the required features and requirements.

Agent	File	Responsibility
Architect	architect.py	Designs the overall system architecture, including the data model and API contract.
Backend	backend.py	Generates the backend codebase: FastAPI routes, SQLAlchemy models, authentication logic.
Frontend	frontend.py	Generates the frontend codebase: React components, TypeScript types, Tailwind styling.
Testing	testing.py	Planned agent for automatically generating test suites (not yet implemented).
Docker	docker.py	Planned agent for generating Docker/containerization configs (not yet implemented).

4.1 Extensibility

Adding a new agent is a deliberately simple, documented process:

1. Create a new file inside agents/.
2. Inherit from BaseAgent.
3. Implement execute() and get_system_prompt().

This plug-in style is exactly what makes the Testing and Docker agents realistic near-term additions rather than a major rewrite. Adding a new test follows the same philosophy — create a file inside tests/ and follow the existing test pattern.

5. Supporting Modules

Module	Purpose
core/config.py	Centralized configuration for the system, including Ollama connection and model settings shared across all agents.
builder/file_builder.py	Takes the code generated by the agents and physically writes it to disk in the correct folder/file structure, turning agent output into a real, runnable project.
memory/	Reserved for a future memory / context-persistence layer (currently a placeholder, not yet functional).
utils/	General-purpose helper utilities used across the codebase.

6. Technology Stack

6.1 Languages

Going by the repository's own language breakdown: Python (77.4%), TypeScript (15.9%), CSS (4.8%), Batchfile (0.8%), Shell (0.5%), JavaScript (0.4%), HTML (0.2%).

6.2 Core Technologies

Layer	Technology
AI / LLM engine	Ollama, running the llama3.2 model locally
Orchestration language	Python 3.11 or 3.12
Generated backend	FastAPI + SQLAlchemy + SQLite
Authentication	JWT tokens with bcrypt password hashing
Generated frontend	React + TypeScript + Tailwind CSS (built with Vite)
Web UI backend	FastAPI with WebSocket support for live progress updates
Web UI frontend	React + TypeScript, Node.js 18+
API documentation	Auto-generated Swagger / OpenAPI docs

7. Project Structure

7.1 Repository Layout

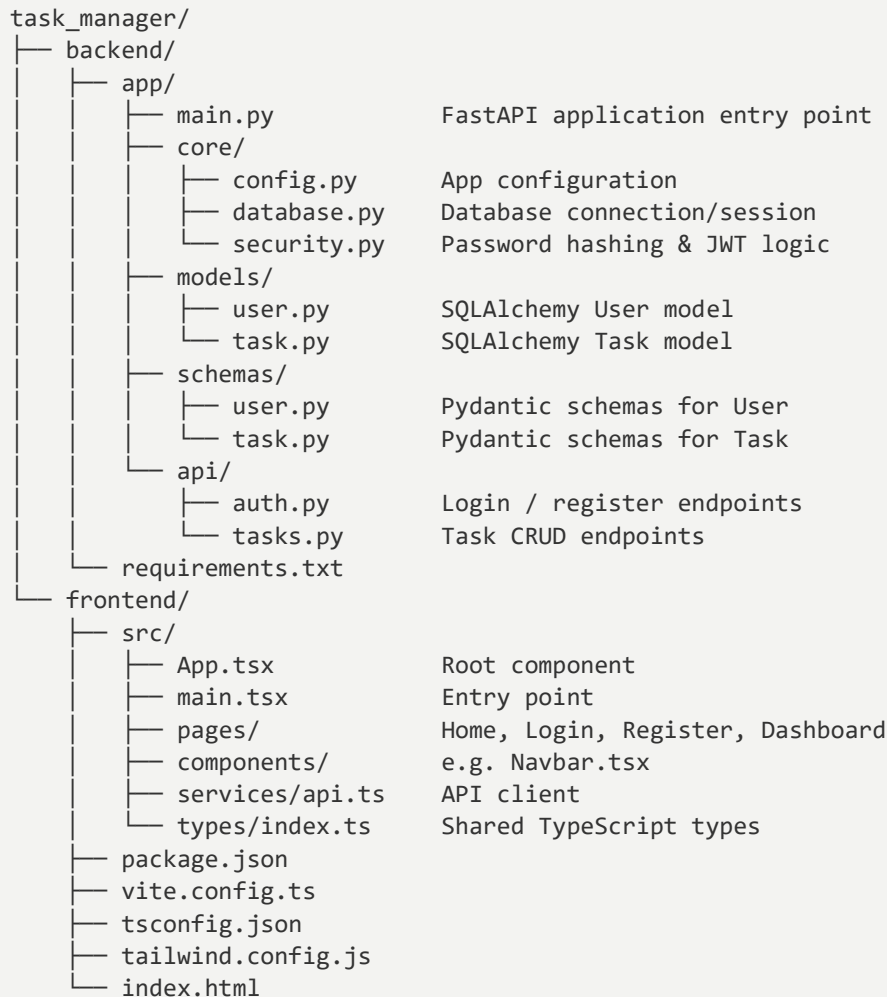
```

multi-agent-swe/
├── agents/      Planner, Architect, Backend, Frontend, Testing, Docker (7 files)
├── core/        Shared configuration – Ollama & system settings (2 files)
├── builder/     Writes generated code to disk as a real project (1 file)
├── memory/      Reserved for future context/memory persistence
├── utils/       Helper utilities
├── ui/
│   ├── backend/ FastAPI + WebSocket server, generated output/, logs/
│   └── frontend/ React + TypeScript + Tailwind web interface
├── output/      Projects generated via the CLI
├── tests/       Unit and full-system tests (11 files)
├── scripts/     Convenience run scripts – run_backend.bat, run_frontend.bat
├── docs/        Full project documentation set (8 files)
├── main.py      CLI entry point
├── requirements.txt
├── README.md
├── CONTRIBUTING.md
└── .env.example

```

7.2 Example: Structure of a Generated Project

Every project produced by the pipeline follows a consistent, predictable structure. Generating a “Task Manager” project, for instance, produces:



8. Key Features

- Automatic backend generation using FastAPI and SQLAlchemy.
- Automatic frontend generation using React, TypeScript, and Tailwind CSS.
- Built-in user authentication using JWT tokens and bcrypt password hashing.
- SQLite database provisioning out of the box.
- Auto-generated, interactive API documentation (Swagger / OpenAPI).
- Production-oriented, professional UI styling in generated frontends.
- Real-time generation progress via WebSocket in the web UI.
- Zero inference cost — runs entirely on a local Ollama model rather than a paid API.

9. Installation & Usage Workflow

9.1 Requirements

- Python 3.11 or 3.12 (3.14 is explicitly not supported)
- Node.js 18+
- Ollama, with the llama3.2 model pulled locally

9.2 Setup

```
# 1. Install Python dependencies
pip install -r requirements.txt

# 2. Install Ollama (see docs/OLLAMA_SETUP.md)

# 3. Pull the model
ollama pull llama3.2

# 4. Install frontend dependencies
cd ui/frontend
npm install
```

9.3 Running via the Web UI (recommended)

```
# Backend
cd ui/backend
python app.py

# Frontend (separate terminal)
cd ui/frontend
npm install
npm run dev

# Then open: http://localhost:3000
```

9.4 Running via the CLI

```
python main.py
```

9.5 Example End-to-End Use

4. Open <http://localhost:3000> in the browser.
5. Enter a project name, e.g. “Task Manager”.
6. Enter a short description, e.g. “An application for managing tasks”.
7. List desired features, e.g. user authentication, task creation/management, task categorization.
8. Click “Generate Project” and wait roughly 5–10 minutes.
9. Run the generated project's backend and frontend to see the finished application.

9.6 Running a Generated Project

```
cd ui/backend/output/task_manager

# Backend
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload

# Frontend
cd frontend
npm install
npm run dev
```

9.7 Verifying a Generated Project

Before running a freshly generated app for the first time, it's worth doing a short manual sanity check:

10. Check the backend models: `head -5 backend/app/models/user.py` should show SQLAlchemy imports.
11. Check password security: `grep "get_password_hash" backend/app/api/auth.py` should show hashed passwords being used, never plaintext.
12. Check the frontend API URL: `grep "API_BASE_URL" frontend/src/services/api.ts` should point to `http://localhost:8000/api`.
13. Check the frontend build config: `ls frontend/` should include both `vite.config.ts` and `index.html`.

9.8 Manual Testing Checklist (Generated App)

14. Open `http://localhost:3000`.
15. Register a new account.
16. Log in.
17. Create a new item (e.g. a task).
18. Delete an item.

10. Testing

The project includes a dedicated `tests/` directory (11 files total) with tests that target individual agents as well as the system as a whole:

Test file	Purpose
<code>test_planner.py</code>	Tests the Planner agent in isolation.
<code>test_architect.py</code>	Tests the Architect agent in isolation.
<code>test_backend_and_builder.py</code>	Tests backend code generation and the file builder together.

Test file	Purpose
test_frontend.py	Tests the Frontend agent in isolation.
test_agents_quick.py	A fast smoke test across all agents.
test_full_system.py	Full end-to-end system test.
test_full_workflow.py	Tests the complete generation workflow.
test_setup.py	Verifies environment/setup correctness.

Running the tests

```
python tests/test_agents_quick.py # quick smoke test
python tests/test_full_system.py  # full system test
python tests/test_setup.py        # environment check
```

Additional detail on the test suite lives in tests/README.md.

11. Results & Performance

Based on the metrics tracked during development, the system currently achieves:

Metric	Value
Success rate	≈ 93%
Generation time	5–10 minutes per project
Inference cost	Zero (runs on local Ollama model)
Manual fixes needed (current)	0

11.1 Before vs. After Recent Refinements

SUMMARY.md documents a clear before/after comparison following a round of reliability fixes to the generation pipeline:

Aspect	Before	After
Manual fixes required	6+ fixes needed per project	0 fixes needed
Time to get a project running	30+ minutes	≈ 2 minutes
Password storage	Stored unhashed	Hashed with bcrypt
UI design quality	Basic / minimal styling	Professional, production-oriented styling
Frontend API URL	Incorrect	Correct (http://localhost:8000/api)

These fixes specifically targeted four things: making sure every generated SQLAlchemy model includes its full set of imports, switching password storage over to bcrypt hashing, correcting the frontend's API base URL, and completing the Vite + TypeScript build configuration so generated frontends actually build out of the box.

12. Documentation Set

The repository keeps a fairly complete internal documentation set, both at the root level and inside docs/:

12.1 Root-Level Documentation

File	Content
README.md	Quick-start guide.
PROJECT_STRUCTURE.md	Full breakdown of the repository's folder structure and data flow.
SUMMARY.md	Comprehensive project summary, including statistics and before/after comparisons.
ORGANIZATION_SUMMARY.md	Summary of how the repository is organized.
QUICK_START.md	Condensed getting-started instructions.
VERIFICATION.md	Verification steps/checklist for the system.
CONTRIBUTING.md	Guide for contributing to the project.

12.2 docs/ Folder

File	Content
docs/USE_WEB_UI.md	Guide for using the web interface.
docs/OLLAMA_SETUP.md	Instructions for installing and configuring Ollama.
docs/CODEBASE_INDEX.md	Full documentation of the codebase.
docs/ARCHITECTURE_VISUAL.md	Visual diagrams of the system architecture.
docs/API_DOCUMENTATION.md	Documentation of the generated API.
docs/DEPLOYMENT_GUIDE.md	Guide for deploying the system.
docs/PROJECT_PLAN.md	The project's original plan.
docs/SUMMARY_AR.md	Arabic-language project summary.

13. Limitations & Future Work

13.1 Current Limitations

- The Testing and Docker agents are planned but not yet implemented, so generated projects don't yet include automated tests or containerization by default.
- The memory/ module for cross-session context persistence is still a placeholder and not functional yet.
- Generation quality depends on the local LLM (llama3.2 via Ollama), which is smaller than large cloud-hosted models and can occasionally produce inconsistent output — which is reflected in the reported ~93% (not 100%) success rate.
- Scope is currently limited to one architectural pattern: FastAPI/SQLAlchemy + React/TypeScript/Tailwind with JWT auth and SQLite.

13.2 Possible Future Directions

- Implement the Testing agent to auto-generate unit/integration tests for generated projects.
- Implement the Docker agent to auto-generate containerization configs for easier deployment.
- Add support for additional database backends (e.g. PostgreSQL) and frontend frameworks.
- Build out the memory module so the system can learn from and reference prior generations.
- Support larger or alternative local models for improved generation accuracy.

14. Conclusion

This project is a practical, working example of a multi-agent AI system applied to a real software engineering task. By splitting planning, architecture, backend development, and frontend development across specialized agents, and by running the whole thing on a free, local LLM through Ollama, Multi-Agent Coordinator can turn a short natural-language description into a functioning full-stack application in a matter of minutes. Some pieces — the testing and containerization agents, the memory module — are still on the roadmap, but the current implementation already provides a complete, end-to-end pipeline: from user input, through both a CLI and a web interface, to a runnable generated project. As a demonstration of applied multi-agent orchestration, it does what it set out to do.

15. Reference

Source code repository:

<https://github.com/abdallared/multi-agent-swe>

License: MIT License. Version: 1.0.